

440_hw1_report

Kanran Peng kp942
Dhruv Kevadlya dak266

February 2025

1 Part 0

The code will generate 50 grid worlds of size 101*101 by another Python code called Maze. After the loop of `setting_up_environment()`, there will be a list of list `result_list` of 50 elements, each element will contain a 2d list representing the maze, with 1 means blocked, and 0 means walkable.

The Maze class will generate a maze by DFS, which will randomly pick a beginning point, and 30% mark a location 0, as walkable; 70% mark this location 1, as blocked. If the DFS cannot go over the whole graph, the maze will call another DFS from another location which is not been visited yet, until all the locations have been visited, which means the whole graph has been initialized as 1 and 0.

2 Part 1

2.1 a)

Short answer: because the agent does not know any of those blocked cells, A* returns a path that goes straight from A to T. And the first is moving east.

When the A* search first searches on start positing A, E2, the agent have no knowledge of the graph, it will look at the 3 locations around E2, which are D2, E1, E3. And compute the f value of all three of them. All three of them will have the same $g = 1$, and use the Manhattan distance to compute the h values.

D2, $h = 4$, $f = 5$

E3, $h = 2$, $f = 3$

E1, $h = 4$, $f = 5$

Among those 3 f values, the agent will use a priority queue, the `open_list`, to store them. And then pop the one with the least f, as the new location, which is E3. And E2 will be put into the `closed_list`.

A* will move to location E3 and compute its neighbor's f value. Since the agent moved 1 step, and another step to reach those new neighbors, it will use $g = 2$ for its neighbor. The result of f values are:

D3: $f = 2 + 3 = 5$

E2: is in the closed_list, skip it

E4: $f = 2 + 1 = 3$

Those 2 locations will be pushed into the open_list.

Then, we will pop the least f from the open list, which is E4, and check all its neighbors:

D4: $f = 3 + 2 = 5$

E3: is in the closed_list, skip it

E5: $f = 3 + 0 = 3$

Those 2 locations will be pushed into the open_list.

Then, pop the least f, which is E5, and it is the destination. A* search is done, return the path: E2, E3, E4, E5.

Then the agent has a path, it will move along it and update its knowledge until the path is blocked by an obstacle. So it moves east to E2 and then finds out the path is blocked, the agent will call A* search again on location E3.

2.2 b)

Short answer: The agent will not call A* search in the same location twice, if it call A* on every free cell, it has called num_all_free_cells times. A path return by A* is at most num_all_free_cells long. The number of movements of the agent is bound by the product of these two numbers.

If we say the target is not reachable, it means after we did some A* search, moved around the maze, and got some new knowledge of blocked locations, the next A* search is not able to reach the target. In this failed A* search, it will return fail if it has popped all the locations from the open_list, which included both open cell and unknown cells, and still has not yet gotten to the destination. In other words, in this A* search, it has visited all the possible moveable locations but none of them is the destination.

If the target is reachable, then every A* search will return a path, and this path's length is at most the number of all walkable cells. If a path is longer than the number of open cells, it means the target is not reachable, because even in the best case, all open cells are in this path, and there is still a lack of some open cells for the agent to reach the target. This means, that after the agent gets a path from A*, it will walk at most number of open cells steps.

Second, we want to make sure that A* search is called at most the number of open cells times. We claim that the agent will not be blocked in the same location twice. If the agent walks along a path, and then it is blocked, it will have the knowledge of the neighbors of the last walkable cell of the path, call it W for convenience. The agent will then do an A* search starting at W, this is the first and last time the agent does an A* search starting at W. We can prove by contradiction, suppose the agent, later on, did another A* search at W, it means before this new A* search, the previous A* search gave a path which is blocked by some neighbors of W. And we know that the only reason A* search gave a blocked path is that the agent does not have the knowledge of the the blocked cells around the last open cell, in this case, it is W again. This contradicts “the agent already explored all neighbors around W, at the first time it arrived at W”. Hence, by contradiction, we proved there will never be two A* searches called on the same location. This means we can call A* search at most number of open cells times.

Combining first and second, we can do at most `num_open_cells` times A* search, and each A* search will give back a path for the agent to walk, which length at most `num_open_cells`. Then the total step of the agent $\leq \text{num_open_cells} * \text{num_open_cells}$.

3 Part 2

If we break ties with the preference for a larger g value, the program is faster.

Prefer large g: finish in 38.55562400817871s

Prefer small g: finish in 39.79965114593506s

```
repeated_forward: 38.55562400817871 s
repeated_backward: 95.15387272834778 s
repeated_forward_prefer_small_g: 39.79965114593506 s
```

Figure 1: Larger g is faster

We notice that the preference of larger g is faster than smaller g, after running 50 101*101 mazes.

To explain this, we can see it as prefer larger g means prefer smaller h, and prefer smaller g means prefer larger h. In the case of ties, if we choose to expand the node which smaller than h, it means we are expanding the one that is closer to the destination. If there is no obstacle between those 2 nodes to the destina-

tion, then the smaller h one will reach the destination within fewer expanding. If there are obstacles, then it does not matter which we prefer, the only thing that matters is which node is blocked by the obstacle, then the blocked one will have a worse path. But if the obstacle blocks both nodes at the same rate with each move, the smaller h one is closer to the destination, it has fewer moves to make to reach the destination, so it will have a lower rate of being blocked by an obstacle.

Hence, preferring a larger g can make the algorithm run faster.

4 Part 3

The `repeated_backward_search` is almost the same as `repeated_forward_search` in implementation, but much slower when running it. It will call A* but exchange the start and dest, so A* will return a path from dest to start. And then the agent will move along this path backward.

```
repeated_forward: 39.039429903030396 s, explored 396998 nodes
repeated_backward: 113.19463920593262 s, explored 708231 nodes
repeated_forward_prefer_small_g: 38.245566844940186 s, explored 396998 nodes
```

Figure 2: backward much slower

After running all 50 mazes, repeated forward took 39.039429903030396 seconds, and expanded 396998 cells. But repeated backward took 113.19463920593262 seconds and expanded 708231 cells. Backward took over 2 times more time and expanded 2 times more cells.

The reason for this is because:

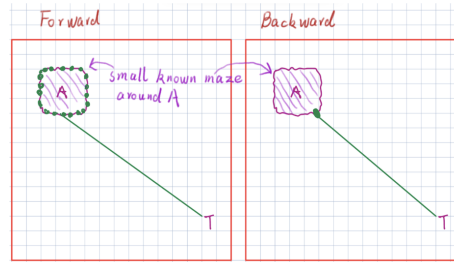


Figure 3:

After we have done some A* search and the agent has moved around to discover the environment around it, the knowledge of the agent becomes like

this, there is a smaller known_maze around it, and all the unknown areas are walkable. Now we call A* search again. In the forward search, if the agent can get out of the small maze around it, it will go to the destination using the shorter Manhattan distance. But for backward search, A* will expand a path from the destination toward the agent, with the shortest Manhattan distance, it will arrive at the small maze around the agent. Now, it will look for only one spot which is the agent's location. The forward search is looking for any spot outside the small maze, and there are many more options available.

For forward search, if we can get to any of those green spots, which means any connection between the agent and any of those green spots is fine, we will be able to reach the target pretty fast. But for backward, we arrive at one single spot on the boundary of the small maze, and the only way to reach the agent is to find a connection between those 2 spots. There are much higher possibilities for the forward search to get out of the small maze.

5 Part 4

5.1 a

Consistent: For all states n , m , let the cost from n to m is $C(n, m)$. The,

$$h(n) \leq C(n, m) + h(m)$$

$h(x)$ is the heuristics function from state x to the destination, in this case, we define it as the Manhattan distance between state x and destination.

$C(n, m)$ is the cost from state n to state m , which means if there is a path from n to m , then the length of the path is this cost.

We can prove "if we use Manhattan distance for $h(x)$ the Inequation will still be true for all n , m belong to the states" by contradiction.

First, let's assume the inequation does not hold, which means $h(n) < C(n, m) + h(m)$. We can understand this inequation as, the Manhattan distance from n to destination, is longer than the (the length of the path from n to m + the Manhattan distance from m to destination), which means if we travel from n to m and then the to destination will end up with a shorter distance. However, according to the definition of Manhattan distance, there does not exist any path from n to destination with a smaller distance than the Manhattan distance. We have found a contradiction.

Hence, by contradiction, we have proved, that the Manhattan distance is consistent within such a grid world.

5.2 b

Short answer: To show that $hnew(n) \leq C(n, m) + hnew(m)$, is showing $gd(s, d) - g(n) \leq C(n, m) + h(m)$. We can set up a contradiction, $gd(s, d) - g(n) < C(n, m) + hnew(m)$, it argues if we go from n to m , then from m to destination is a shorter path. It is shorter than going directly from n to destination according to some path discovered by another A^* search before. It contradicts to A^* search will always give the shortest path.s

If we use $hnew(n)$ as the renewed_h value for a state n . The state n is a state that expanded in the last A^* search, it will have $g(n)$, $h(n)$. The $g(n)$ means the cost from the location that the last A^* search begins to this state n , and $h(n)$ means the heuristics from n to destination. The $hnew(n)$ is computed as, $gd(s, d) - g(n)$, where $gd(s, d)$ means the length of the path returned from A^* , which is the smallest path length from the starting point to the destination.

We can understand the $hnew(n)$ like this, after the A^* search is done, it will expand some nodes, each of them is associated with an g value. We can group all the nodes that have the same g value together, and give them a new $hnew$ value. If there is a group that has $g = x$, means all those nodes are discovered within x steps from the starting node, and we know there is at least one of them on the shortest path from the starting node to the destination. If no node in this group is also in the path, we say that the path does not contain a node that is x step away from the starting node, this means the A^* search failed. Now we will update all the nodes' h value in group x , to the length path - x , this value means there is at least one of the nodes in this group can reach the destination with (length path - x) steps, and all the other nodes in this group will be able to reach the destination with non-less steps. This is because A^* search will find the shortest path, if any nodes in the group can reach the destination with fewer steps, it will be in the path. Then we can prove admissible, for all nodes in group x , the path from them to the destination must be greater or equal to the shortest path among all of them to the destination.

This is admissible even if the agent has moved around and discovered some new environment. After the agent discovers new environment, there are two opinions, the shortest path from one node in group x is blocked, or the path is not. If the path is not blocked, then $hnew$ for all nodes in group x does not change; if the path is blocked, then we can only find a new path from some node in group x with a longer length.

To show that $hnew(n) \leq C(n, m) + hnew(m)$, is showing $gd(s, d) - g(n) \leq C(n, m) + h(m)$. We can set up a contradiction, $gd(s, d) - g(n) < C(n, m) + hnew(m)$, it argues if we go from n to m , then from m to destination is a shorter path, shorter than go directly from n to destination. But as we stated before, there is no such node in group $g(n)$ steps away from the starting node that can reach the destination with fewer steps than $hnew(n)$, which is a contradiction.

6 Part 5

```
repeated_forward: 39.039429903030396 s, explored 396998 nodes  
repeated_backward: 112.19455220592262 s, explored 788231 nodes  
repeated_forward_prefer_small_g: 38.245566844940186 s, explored 396998 nodes  
Adaptive A* search: 36.88574194908142 s, explored 362684 nodes
```

Figure 4:

The Adaptive A* search is faster than the A* search and it expanded fewer cells.

The repeated_forward A* took 39.039429903030396 seconds and explored 396998 nodes.

The Adaptive A* took only 36.88574194908142 seconds and explored 362684 nodes.

This is because, for Adaptive, the h_{new} value of a state n is greater than the Manhattan distance from n to destination. Suppose there are 2 nodes, n , m . Node n has a lower Manhattan distance to the destination but a very larger $h_{new}(n)$.

Node m has a larger Manhattan distance to the destination but a very lower $h_{new}(m)$.

The lower the h_{new} value means there is a larger possibility that the node m is pretty close with the destination node. And also node n has a larger possibility of being far from the destination. In repeated forward search, it will first expand node n , but in Adaptive search, it will expand node m first. Because m has a larger possibility of being close to the destination, expanding m first is a better choice.

7 Part 6

If we want to know, does the repeated forward prefer larger g faster than the repeated forward prefers small g ? We can first give a hypothesis based on the experiment, $h1$, preference of larger g is faster. In contrast, $h0$, preference of smaller g is faster.

And then we repeated the running our code a large number of times, for example, 100 times. After that, we check how many times $h1$ is satisfied, and how many times $h0$ is satisfied. If we $h0$ happened much less than $h1$ happened, we can conclude that $h1$ does have a larger chance of happening. If we do not observe that $h1$ happens sufficiently more than $h0$, then we cannot conclude that $h1$ has a larger chance of happening.